

Database System Engineering and Distributed Backend Development

Course Code: 25CS1302E

Week -3

Scenario: BookFlow wants to capture user reviews and technical specs for books, where every book has different details. A fixed SQL table is too restrictive, so MongoDB is used to manage this unstructured content and generate a "Top Rated" report using the aggregation pipeline.

Tasks: 1) Document Modeling — create a book_metadata collection with embedded patterns (reviews list inside the book document) and referenced patterns (linking to member_id in SQL). 2) Advanced Querying — use comparison operators (\$gt, \$in) to find books rated above 4, and a regex query to find books with "Digital" formats. 3) Aggregation Pipeline — match books published after 2020, \$unwind the reviews array, then \$group by title and compute the average rating. 4) Indexing for Speed — create a text index on the review comments field and use .explain() to check whether the query uses the index.

Skills gained: BSON modelling and embed-vs-reference decisions, pipeline engineering with \$match/\$group/\$unwind, multikey/text index performance tuning, and handling polymorphic documents.

Task 1: Document Modelling

The problem with a fixed SQL table

In a relational model, every book would need a row in a books table with a fixed set of columns: pages, file_size, narrator, runtime, binding_type, drm_flag, and so on. Since a hardcover doesn't have a narrator and an audiobook doesn't have page count, most of these columns would be NULL for any given row. As BookFlow adds new formats (for example, a future "interactive ebook" with different specs again), the table needs a schema migration (ALTER TABLE) every time. This is the classic impedance mismatch between rigid relational schemas and variable, semi-structured content.

Why MongoDB fits

MongoDB stores data as BSON documents (a binary superset of JSON) inside collections, and documents in the same collection are not required to share the same fields. This is called a flexible or polymorphic schema. Each book document only carries the fields relevant to its own format — no wasted NULL columns, no migrations needed to add a new book type.

The core modelling decision: embed vs. reference

MongoDB gives you two ways to represent relationships between data:

Approach	What it means	Best for
Embedding	Store related data as a nested object/array inside the parent document	Data always accessed together, bounded size, and "owned" by the parent (composition)
Referencing	Store an ID that points to a document in another collection (or database)	Large, unbounded, or shared data owned by a different system of record

Reviews are embedded because:

- A review only makes sense in the context of its book — you almost never query reviews independently of the book they belong to.
- The array is bounded in practice; if it weren't, you'd reconsider (see below).
- Embedding lets you fetch the book and all its reviews in a single read, with no extra round trip — a major performance win, since MongoDB documents are the unit of atomic read/write.

Member identity is referenced (member_id) because:

- Member data (name, email, address, account status) is owned by the existing SQL system — the authoritative source of truth.
- Duplicating member details inside every review would mean that a name change requires updating every review the member ever wrote — a consistency nightmare.
- Storing just the member_id keeps the two systems loosely coupled: the app layer can join review data with member data at read time without MongoDB needing to know anything about member internals.

When embedding would break down

If reviews could grow unbounded (say, a viral book gets 500,000 reviews), embedding becomes risky because MongoDB documents have a 16MB size limit, and frequently updating a large embedded array causes the whole document to be rewritten, hurting write performance. In that case, you would reference reviews instead — store them in a separate reviews collection with a book_id field pointing back to the book. This lab uses embedding because review counts per book are modest and always read together with the book.

Polymorphic specs field

The specs sub-document is intentionally different per book (pages for print, narrator/runtime_min for audio, fileSize_MB/drm for digital). This is possible precisely because MongoDB doesn't enforce a fixed schema — each document can carry its own shape of specs.

Sample document

```
db.book_metadata.insertOne({
  title: "The Silent Algorithm",
  author: "R. Menon",
  genre: "Technology",
  publishedYear: 2022,
  formats: ["Hardcover", "Digital", "Audiobook"],
  specs: {
    pages: 340,
    fileSize_MB: 4.2,
    drm: true,
    narrator: "K. Rao",
    runtime_min: 610
  },
  avgRating: 4.6,
  reviews: [
```

```
{
  member_id: "M1001",
  rating: 5,
  comment: "Excellent deep dive into digital systems design.",
  date: ISODate("2023-01-15")
},
{
  member_id: "M1042",
  rating: 4,
  comment: "Good but the Digital edition had formatting issues.",
  date: ISODate("2023-02-20")
}
]
});
```

db.book_metadata.insertMany([

```
{
  title: "Ocean of Code",
  author: "A. Fernandes",
  genre: "Technology",
  publishedYear: 2021,
  formats: ["Digital", "Paperback"],
  specs: { pages: 210, fileSize_MB: 2.1, drm: false },
  avgRating: 4.2,
  reviews: [
    { member_id: "M2001", rating: 4, comment: "Solid Digital read, clean layout.", date:
ISODate("2022-05-10") },
    { member_id: "M2002", rating: 5, comment: "Best programming book this year.", date:
ISODate("2022-06-01") }
  ]
},
{
  title: "Whispers of Yesterday",
  author: "S. Kapoor",
  genre: "Fiction",
  publishedYear: 2019,
```

```
formats: ["Hardcover", "Paperback"],
specs: { pages: 410, binding: "Sewn" },
avgRating: 3.8,
reviews: [
    { member_id: "M3001", rating: 3, comment: "Slow start but strong ending.", date: ISODate("2020-01-15") },
    { member_id: "M3002", rating: 4, comment: "Beautifully written prose.", date: ISODate("2020-02-11") }
],
},
{
    title: "Quantum Horizons",
    author: "R. Menon",
    genre: "Science",
    publishedYear: 2023,
    formats: ["Digital", "Audiobook"],
    specs: { fileSize_MB: 5.6, drm: true, narrator: "P. Iyer", runtime_min: 540 },
    avgRating: 4.9,
    reviews: [
        { member_id: "M4001", rating: 5, comment: "The Digital format made annotations so easy.", date: ISODate("2023-08-01") },
        { member_id: "M4002", rating: 5, comment: "Mind-bending and well narrated.", date: ISODate("2023-09-14") }
    ]
}
];
```

Task 2: Advanced Querying

Why comparison and array operators matter

MongoDB's query language extends simple equality matching with operators for ranges, set membership, and pattern matching — critical for a catalog where fields like formats are arrays, not single values.

\$gt (greater than)

```
db.book_metadata.find({ avgRating: { $gt: 4 } });
```

This scans (or, with an index, seeks) for documents where avgRating exceeds 4. Internally, if a B-tree index exists on avgRating, MongoDB can jump directly to the first value greater than 4 rather than scanning every document — the same idea as WHERE avgRating > 4 in SQL, but expressed as a JSON-style query document.

\$in (set membership)

```
db.book_metadata.find({ formats: { $in: ["Digital", "Audiobook"] } });
```

Because formats is an array field, MongoDB automatically checks whether any element of the array matches any value in the \$in list. This is equivalent to SQL's WHERE format IN (...), except it also transparently handles the fact that a single book can belong to multiple formats at once — something a single-valued SQL column couldn't represent without a join table.

Regex matching

```
db.book_metadata.find({ formats: { $regex: "Digital", $options: "i" } });
```

\$regex lets you match partial or pattern-based text rather than requiring an exact string. The "i" option makes it case-insensitive, so "digital", "Digital", and "DIGITAL" all match — useful when format naming isn't perfectly standardized.

A second regex example — searching reviews.comment for "Digital" — demonstrates dot notation, MongoDB's way of reaching into nested/embedded fields (including fields inside array elements) without needing a join. Projecting "reviews.\$": 1 then returns only the first matching review element, rather than the entire array, keeping the response focused.

Why this matters for BookFlow

These operators let the application answer real product questions directly at the database layer — "show me highly rated books," "show me anything available digitally," "find reviews mentioning digital issues" — without pulling every document into application code and filtering there, which would not scale.

a) Books rated above 4 (\$gt)

```
db.book_metadata.find(  
  { avgRating: { $gt: 4 } },  
  { title: 1, avgRating: 1, _id: 0 }
```

);

b) Books available in specific formats (\$in)

```
db.book_metadata.find(  
  { formats: { $in: ["Digital", "Audiobook"] } },  
  { title: 1, formats: 1, _id: 0 }  
);
```

c) Combine both with \$and (implicit)

```
db.book_metadata.find(  
  avgRating: { $gt: 4 },  
  formats: { $in: ["Digital"] }  
));
```

d) Regex query — books with "Digital" format

```
db.book_metadata.find(  
  { formats: { $regex: "Digital", $options: "i" } },  
  { title: 1, formats: 1 }  
);
```

e) Regex query — reviews mentioning "Digital" in comments (nested field)

```
db.book_metadata.find(  
  { "reviews.comment": { $regex: "Digital", $options: "i" } },  
  { title: 1, "reviews.$": 1 }  
);
```

Task 3: Aggregation Pipeline — "Top Rated" Report

What an aggregation pipeline is

MongoDB's aggregation framework processes documents through a sequence of stages, where each stage transforms the data and passes it to the next — conceptually similar to Unix pipes (`cmd1 | cmd2 | cmd3`), or a SQL query broken into `WHERE` → `JOIN/explode` → `GROUP BY` → `ORDER BY` steps.

Stage-by-stage breakdown

1. `$match` — filter early

```
{ $match: { publishedYear: { $gt: 2020 } } }
```

Functionally like a SQL `WHERE` clause. Placing `$match` first is a deliberate performance choice: it shrinks the working set before the more expensive stages run, and if a supporting index exists on `publishedYear`, this stage can use it instead of scanning the whole collection.

2. `$unwind` — flatten the array

```
{ $unwind: "$reviews" }
```

Each book document contains an array of reviews. `$unwind` deconstructs that array: for a book with 3 reviews, one input document becomes 3 output documents, each with `reviews` replaced by a single review object instead of the whole array. This is necessary because the next stage (`$group`) needs to operate on individual rating values, not a whole array at once. Conceptually, `$unwind` is similar to a SQL `CROSS JOIN / LATERAL VIEW explode()` operation on a nested collection.

3. `$group` — aggregate per title

```
{
  $group: {
    _id: "$title",
    avgReviewRating: { $avg: "$reviews.rating" },
    totalReviews: { $sum: 1 },
    author: { $first: "$author" }
  }
}
```

This is the equivalent of SQL's `GROUP BY title`. The `_id` field defines the grouping key. `$avg` computes the mean of `reviews.rating` across all flattened review documents sharing the same title. `$sum: 1` counts how many review-documents fell into each group. `$first` carries the author's name through, since it's the same for every review of a given title.

4. `$sort` — order by rating

```
{ $sort: { avgReviewRating: -1 } }
```

`-1` means descending, so the highest-rated books surface first — producing the actual "Top Rated" ordering the report needs.

5. `$project` — reshape output

```
{
  $project: {
    _id: 0,
    title: "$_id",
    author: 1,
    avgReviewRating: { $round: ["$avgReviewRating", 2] },
    totalReviews: 1
  }
}
```

\$project controls the final shape of each output document — renaming `_id` back to `title` (since \$group renamed it), rounding the average to 2 decimal places for readability, and dropping internal fields the client doesn't need.

Why the order of stages matters

Aggregation stages run sequentially, and each one's output becomes the next one's input. Filtering with \$match before \$unwind means you never bother exploding review arrays for books outside the date range — a meaningful efficiency gain at scale. Putting \$sort after \$group (rather than before) ensures you're sorting the final aggregated averages, not the raw per-review documents.

```
db.book_metadata.aggregate([
  // Stage 1: filter books published after 2020
  { $match: { publishedYear: { $gt: 2020 } } },

  // Stage 2: deconstruct the reviews array — one output doc per review
  { $unwind: "$reviews" },

  // Stage 3: group by title, compute average rating & review count
  {
    $group: {
      _id: "$title",
      avgReviewRating: { $avg: "$reviews.rating" },
      totalReviews: { $sum: 1 },
      author: { $first: "$author" }
    }
  },
])
```



```
// Stage 4: sort descending by average rating
{ $sort: { avgReviewRating: -1 } },

// Stage 5: reshape output
{
  $project: {
    _id: 0,
    title: "$_id",
    author: 1,
    avgReviewRating: { $round: ["$avgReviewRating", 2] },
    totalReviews: 1
  }
}
]);
```

Expected output shape:

```
[
  { title: "Quantum Horizons", author: "R. Menon", avgReviewRating: 5.0, totalReviews: 2 },
  { title: "Ocean of Code", author: "A. Fernandes", avgReviewRating: 4.5, totalReviews: 2 }
]
```

Task 4: Indexing for Speed

Why indexes matter

Without an index, MongoDB must perform a collection scan (COLLSCAN) — reading every single document to check if it matches the query. This is $O(n)$ and gets slower as the collection grows. An index is a separate, sorted data structure (typically a B-tree) that lets MongoDB find matching documents in roughly $O(\log n)$ time instead.

Text index

```
db.book_metadata.createIndex({ "reviews.comment": "text" });
```

A text index is a special MongoDB index type designed for full-text search. Rather than storing the exact comment string, it tokenizes the text (splitting into words), removes common "stop words" (like "the", "a", "is"), and applies stemming (so "running" and "run" are treated as related). This lets you search for concepts/words rather than exact substrings.

```
db.book_metadata.find({ $text: { $search: "Digital" } });
```

The `$text` operator queries against the text index and can also return a relevance score (`$meta: "textScore"`) indicating how well each document matches the search terms — letting you sort results by relevance, similar to how a search engine ranks pages.

Verifying index usage with `.explain()`

```
db.book_metadata.find({ $text: { $search: "Digital" } }).explain("executionStats");
```

The important fields to check in the output:

- `winningPlan.stage` — should read `TEXT` (confirming the text index was used) rather than `COLLSCAN` (full scan).
- `executionStats.totalDocsExamined` vs. `nReturned` — a well-indexed query examines close to the same number of documents it returns; a poorly indexed one examines far more (potentially the entire collection).
- `executionStats.executionTimeMillis` — lets you directly compare indexed vs. unindexed performance on the same data.

Why regex can't use a standard index efficiently

```
db.book_metadata.find({ "reviews.comment": { $regex: "Digital", $options: "i" } }).explain("executionStats");
```

A regular (non-text) index is a sorted B-tree over field values. It can only be used efficiently for regex patterns anchored at the start of the string (e.g., `^Digital`), because that lets MongoDB do a sorted range seek. A pattern like `"Digital"` with no anchor could match anywhere in the string, so MongoDB can't narrow the B-tree range and falls back to scanning every document — hence `COLLSCAN` in the explain output. This is exactly why a text index is the right tool for keyword search inside comments, while a regular index is better suited to prefix or exact-match lookups.

Multikey indexes

```
db.book_metadata.createIndex({ formats: 1 });
```

When you index a field whose value is an array (like `formats`, or `reviews.rating` inside embedded documents), MongoDB automatically creates a multikey index — internally, it adds one index entry per array element, so a query can match if any element satisfies the condition. This is what makes `$in` and `$gt` queries against array fields fast once indexed, whereas without the index they'd require scanning and inspecting every array manually.

a) Create a text index on review comments

```
db.book_metadata.createIndex({ "reviews.comment": "text" });
```

b) Query using the text index

```
db.book_metadata.find(
  { $text: { $search: "Digital" } },
  { title: 1, score: { $meta: "textScore" } }
).sort({ score: { $meta: "textScore" } });
```

c) Verify index usage with `.explain()`

```
db.book_metadata.find(
  { $text: { $search: "Digital" } }
).explain("executionStats");
```

What to check in the output:

- `winningPlan.stage` should show `TEXT` (not `COLLSCAN`), confirming the text index is used.
- `executionStats.totalDocsExamined` should be much lower than `totalDocsExamined` of a full collection scan.
- `executionStats.executionTimeMillis` for a before/after comparison.

d) Compare against a non-indexed regex scan

```
db.book_metadata.find(
  { "reviews.comment": { $regex: "Digital", $options: "i" } }
).explain("executionStats");
```

This will show stage: `"COLLSCAN"` — a regex without an anchored prefix cannot use a standard index, illustrating why the text index is preferable for keyword search on comments.

e) Supporting index for common queries (multikey index)

Since `formats` and `reviews` are arrays, an index on them becomes a **multikey index** automatically:

```

db.book_metadata.createIndex({ formats: 1 });
db.book_metadata.createIndex({ avgRating: -1 });
db.book_metadata.find({ avgRating: { $gt: 4 } }).explain("executionStats");
// winningPlan.stage should now show IXSCAN instead of COLLSCAN

```

Summary: How the Four Tasks Connect

Task	Concept Demonstrated	Real-World Value for BookFlow
1. Document Modeling	Embedding vs. referencing, polymorphic schema	Handles books with wildly different specs without schema migrations
2. Advanced Querying	\$gt, \$in, \$regex	Supports catalog filtering and search features in the app
3. Aggregation Pipeline	\$match → \$unwind → \$group → \$sort → \$project	Powers the "Top Rated" report directly inside the database
4. Indexing	Text indexes, multikey indexes, .explain()	Keeps queries fast as the review corpus scales into the millions